

Do OpenFlow ao P4: plano de dados programável e priorização de tráfego de sensores IoT em uma provedora de nuvem

Ciro Guilherme Nass

Bacharelado em Ciência da Computação
Instituto Federal do Paraná (IFPR), Câmpus Pinhais
Pinhais, PR, Brasil
20230007547@estudantes.ifpr.edu.br

Abstract. *The cloud provider from the previous assignment now needs to prioritize readings from IoT sensors that speak a proprietary protocol, but its OpenFlow switches only recognize a fixed set of header fields and the vendor takes years to add new ones. This paper explains why OpenFlow reached that limit, describes how the P4 language and the PISA architecture move the definition of the packet format from the chip to the program, gives an example of a P4 program that reads the proprietary sensor header and sends urgent readings to a higher priority queue, and presents ONOS as a controller that speaks OpenFlow and P4Runtime at the same time, which makes a gradual migration possible.*

Resumo. *A provedora de nuvem da atividade anterior precisa agora priorizar as leituras de sensores IoT que falam um protocolo proprietário, mas seus switches OpenFlow só reconhecem um conjunto fixo de campos de cabeçalho e o fabricante leva anos para incluir campos novos. Este trabalho explica por que o OpenFlow chegou a esse limite, descreve como a linguagem P4 e a arquitetura PISA tiram a definição do formato do pacote do chip e a colocam no programa, apresenta um exemplo de programa P4 que lê o cabeçalho proprietário do sensor e envia as leituras urgentes para uma fila de maior prioridade, e discute o ONOS como controlador capaz de falar OpenFlow e P4Runtime ao mesmo tempo, o que viabiliza uma migração gradual.*

1. Introdução

Na atividade anterior, a provedora de nuvem aparecia presa a equipamentos fechados de vários fabricantes, sem conseguir uniformizar a operação nem implantar protocolos próprios. A resposta discutida foi separar o plano de controle do plano de dados e concentrar as decisões em um controlador, que é o que o OpenFlow viabiliza.

O problema que ela traz agora está uma camada abaixo. Mesmo com SDN e OpenFlow em operação, os switches não conseguem tratar de forma diferenciada os pacotes dos sensores IoT, porque o protocolo desses sensores é proprietário e o chip não sabe lê-lo. Nenhuma regra instalada pelo controlador resolve isso, já que a regra só pode falar sobre campos que o hardware reconhece. As seções seguintes tratam das limitações do OpenFlow e do que o P4 propõe no lugar (Seção 2), do funcionamento da arquitetura P4 nesse cenário (Seção 3), de um exemplo de priorização das leituras dos sensores (Seção 4) e do ONOS como controlador para conduzir a transição (Seção 5).

2. Limitações do OpenFlow e a proposta do P4

O OpenFlow foi a primeira interface padronizada entre a camada de controle e a de encaminhamento de uma arquitetura SDN. Ele separou os planos, mas deixou o plano de dados como executor de funções definidas de antemão. A especificação lista os campos de cabeçalho que o switch precisa saber comparar, e o ASIC é fabricado para reconhecer exatamente esses campos: porta de entrada, endereços MAC, tipo Ethernet, identificador de VLAN, endereços IP e portas TCP/UDP [ONF 2015]. O controlador escolhe as regras, mas não escolhe o vocabulário com que elas são escritas. Daí saem as três limitações que interessam ao caso.

A primeira é o conjunto fixo de campos. Um protocolo experimental ou proprietário não existe do ponto de vista do switch. O equipamento até encaminha os pacotes dos sensores olhando IP e UDP, mas tudo que vem depois do cabeçalho de transporte é carga útil sem significado para ele, o que impede qualquer tratamento diferenciado.

A segunda é o crescimento da própria especificação. Para acompanhar novas demandas, cada versão acrescentou recursos: múltiplas tabelas e tabelas de grupo na 1.1, IPv6 e matches extensíveis em formato TLV na 1.2, medidores de banda na 1.3, tabelas de egresso na 1.5. O número de campos de correspondência saltou de 12 na versão 1.0 para mais de 40 na 1.3, e os próprios autores do P4 usam esse salto como sintoma: a especificação ficou complexa e difícil de implementar por inteiro em hardware, e mesmo assim continuou sem permitir cabeçalhos novos [Bosshart et al. 2014]. A versão 1.5, de dezembro de 2014, segue sendo a atual.

A terceira é a dependência do silício. Os mecanismos de busca e encaminhamento vêm gravados no ASIC, então nenhuma regra enviada pelo controlador muda o que o chip sabe analisar. Incluir um campo novo exige uma revisão do padrão e, quase sempre, uma geração nova de equipamentos. É exatamente o atraso de anos que a provedora relata.

O P4 ataca o problema pela outra ponta. No modelo tradicional o software se adapta ao hardware, de baixo para cima; com plano de dados programável a direção se inverte e o software passa a dizer como o hardware deve se comportar. O artigo que propôs a linguagem lista três objetivos que respondem diretamente às limitações acima [Bosshart et al. 2014]. A independência de protocolo faz com que o switch não venha com uma lista de cabeçalhos conhecidos: quem descreve os cabeçalhos e a forma de extraí-los é o programa, de modo que o cabeçalho dos sensores ganha o mesmo status de um cabeçalho IPv4. A reconfiguração em campo permite alterar o processamento depois que o equipamento já está instalado, o que transforma uma compra em uma recompilação. A independência de alvo faz com que o mesmo código sirva para dispositivos diferentes, já que o compilador P4C traduz o programa para as instruções de um ASIC como o Tofino, de um FPGA, de uma placa de rede inteligente ou do switch de software BMv2.

Convém não tratar as duas tecnologias como concorrentes diretas. O OpenFlow é um protocolo de comunicação entre controlador e switch; o P4 é uma linguagem que define o pipeline do switch. O modelo de tabelas continua valendo, e o que muda é quem define a estrutura dessas tabelas. O papel do OpenFlow também mudou: ele segue atendendo equipamentos de pipeline fixo, enquanto a conversa com um switch programável passa a acontecer pelo P4Runtime, cuja interface é gerada a partir do próprio programa P4.

3. A arquitetura P4 no cenário da provedora

O P4 opera sobre a PISA (Protocol-Independent Switching Architecture), um pipeline em que o pacote atravessa entrada, analisador, estágios de match-action de ingresso, buffer, estágios de match-action de egresso e saída. O programa da provedora preencheria essas etapas da seguinte forma.

Os cabeçalhos abrem o programa e definem o formato dos dados, com um nome e uma largura em bits para cada campo. A equipe declararia Ethernet, IPv4 e UDP, mais um cabeçalho para o protocolo dos sensores, com os campos que a documentação do fabricante do sensor descrever.

O analisador é uma máquina de estados que percorre o pacote e move os bits para as variáveis declaradas. Ele começa sempre no estado start, usa o método extract a cada transição e termina em accept ou reject. O caminho seria Ethernet, IPv4, UDP e, quando a porta de destino for a dos sensores, um estado que extrai o cabeçalho proprietário. O pacote que antes era opaco passa a ter campos nomeados e disponíveis para decisão.

As tabelas e as ações concentram a lógica. Cada tabela declara quais campos entram na correspondência e quais ações podem ser executadas em caso de acerto; as ações alteram cabeçalhos e metadados e podem receber parâmetros vindos do plano de controle. A estrutura da tabela é estática, definida no código, mas as entradas são inseridas dinamicamente pelo controlador durante a operação. Criar uma tabela de prioridade para os sensores exige recompilar o programa; decidir quais sensores têm prioridade é uma chamada em tempo de execução.

O buffer separa as duas metades do pipeline e é onde ficam as filas de saída, ou seja, é onde a priorização acontece de fato. Os estágios de egresso já trabalham no contexto da porta de saída. Fechado o pipeline, o pacote é remontado com os cabeçalhos modificados.

O fluxo de trabalho da equipe seria escrever um arquivo provedora.p4, compilá-lo com o P4C e carregar o resultado no equipamento. O compilador gera o binário do alvo e a descrição da API do programa, que informa ao controlador quais tabelas existem e quais parâmetros cada ação espera. Quando aparecer outro protocolo, basta alterar o arquivo, recompilar e recarregar o pipeline, sem abrir chamado com o fabricante. Uma restrição precisa ficar registrada: nada disso funciona nos switches que a provedora tem hoje, porque o P4 exige plano de dados programável. Um caminho razoável é validar o programa no BMv2 com Mininet, colocar equipamento programável nas bordas onde os sensores se conectam e manter o núcleo legado sob OpenFlow durante a transição.

4. Priorização do tráfego dos sensores IoT

Suponha que os sensores enviem leituras em UDP na porta 5683 e que o protocolo proprietário traga, entre outros campos, o identificador do dispositivo e um nível de criticidade. Para um switch OpenFlow, todo esse tráfego é apenas UDP na porta 5683: ou a provedora prioriza todas as leituras, ou não prioriza nenhuma, e um alarme de

temperatura disputa fila com uma medição de rotina. O primeiro passo em P4 é declarar o cabeçalho e ensinar o analisador a extraí-lo.

```
header sensor t {
    bit<16> id_sensor;
    bit<8>  criticidade;
    bit<32> leitura;
}

state parse_udp {
    packet.extract(hdr.udp);
    transition select(hdr.udp.dstPort) {
        5683 : parse_sensor;
        default : accept;
    }
}
```

O segundo passo é criar a tabela que classifica a leitura e a ação que a marca.

```
action marcar_prioridade(bit<3> fila) {
    standard_metadata.priority = fila;
    hdr.ipv4.diffserv = 0xB8;
}

table t_prioridade_sensor {
    key = { hdr.sensor.criticidade : ternary; }
    actions = { marcar_prioridade; NoAction; }
    default_action = NoAction();
    size = 1024;
}
```

O nome do campo que representa a fila depende da arquitetura escolhida: no v1model é `standard_metadata.priority` e na PSA o equivalente é `class_of_service`. Nos dois casos a ação avisa ao gerenciador de tráfego em qual fila o pacote deve entrar, e no buffer a leitura urgente passa na frente do tráfego comum. A escrita no campo `diffserv` cumpre outro papel: 0xB8 corresponde ao DSCP EF, reservado a tráfego sensível a atraso, e os equipamentos tradicionais no caminho não entendem o cabeçalho proprietário mas entendem DSCP. O P4 traduz uma informação que só o switch programável enxerga para algo que o resto da rede respeita.

O terceiro passo fica com o controlador. Se a provedora decidir que criticidade acima de 200 vai para a fila expressa e o restante para a fila padrão, isso são duas entradas instaladas via `P4Runtime`, alteráveis a qualquer momento sem tocar no programa. A correspondência do tipo ternary foi escolhida justamente para permitir faixas de valores por máscara de bits.

Vale notar um ganho que não aparece em OpenFlow. Um register indexado pelo identificador do sensor guarda quantas leituras cada dispositivo enviou em uma janela de tempo, e o próprio switch rebaixa a prioridade de um sensor defeituoso que passa a inundar a rede. Em OpenFlow, essa contagem exigiria enviar pacotes ao controlador por `packet-in` e esperar a resposta, o que não atende ao requisito de tempo real.

5. ONOS como controlador SDN

O ONOS (Open Network Operating System) é um controlador de código aberto escrito em Java e mantido no ecossistema da Open Networking Foundation, projetado para redes de operadoras e provedores de serviço [ONF 2026]. É uma escolha coerente para o caso porque acompanha justamente a transição descrita aqui.

Entre as funcionalidades, a que mais o distingue é o núcleo distribuído: o ONOS roda como um cluster de instâncias que compartilham a mesma visão da rede, de modo que a queda de uma instância não derruba o controle e é possível ganhar capacidade acrescentando nós. Ele descobre e mantém a topologia de dispositivos, enlaces e hosts, e a oferece às aplicações como um grafo, sobre o qual se programa em termos de rede e não de equipamento. As APIs norte aparecem em níveis diferentes: FlowRule, para instalar regras específicas, FlowObjective, para pedidos independentes do pipeline do equipamento, e o framework de intents, em que a aplicação declara o resultado desejado, como conectividade entre dois pontos, e o controlador calcula o caminho e o restabelece sozinho quando um enlace cai. As aplicações são módulos que podem ser instalados e removidos com a rede em operação, e a administração é feita por interface gráfica de topologia, linha de comando ou API REST.

No lado sul, o ONOS trabalha com plugins que reúnem drivers específicos de cada dispositivo e bibliotecas de protocolo compartilhadas, o que permite ao mesmo controlador falar dialetos diferentes com equipamentos diferentes: OpenFlow com switches de pipeline fixo, P4Runtime com alvos programáveis, gNMI para configuração e telemetria, gNOI para operações no equipamento, além de NETCONF e OVSDB. A lista de drivers inclui OVS, BMv2, Barefoot, Cisco, Juniper e Huawei, entre outros.

A vantagem mais relevante para o caso é a convivência. A provedora não precisa trocar a frota inteira de uma vez: os switches antigos continuam recebendo regras por OpenFlow enquanto os programáveis da borda rodam o programa dos sensores e recebem as entradas de tabela por P4Runtime, e a mesma aplicação de controle enxerga os dois pelo grafo da rede. Somam-se a isso o cluster nativo, que evita transformar o controlador em ponto único de falha, e o fato de o projeto ser aberto e aceitar equipamentos de vários fabricantes, o que ataca o aprisionamento tecnológico discutido na atividade anterior. Entre as alternativas, o OpenDaylight também é escrito em Java e tem forte apoio a NETCONF e YANG, e o Ryu é um controlador em Python, mais simples, comum em ensino e em protótipos.

6. Considerações finais

As três limitações do OpenFlow descritas aqui têm a mesma raiz: quem define o formato do pacote é o chip, não o operador. Enquanto o switch só souber comparar os campos que a especificação previu, qualquer protocolo novo depende de revisão do padrão e de silício novo. O P4 desloca essa fronteira, e por isso resolve o caso dos sensores, que nenhuma regra OpenFlow resolveria. Para a provedora, a recomendação prática é escrever o programa P4 com o cabeçalho proprietário, validar o comportamento no BMv2, adotar hardware programável nas bordas onde os sensores se conectam e usar um controlador como o ONOS, que fala com os equipamentos antigos e com os novos enquanto a migração acontece.

Referências

- Bosshart, P., Daly, D., Gibb, G., Izzard, M., McKeown, N., Rexford, J., Schlesinger, C., Talayco, D., Vahdat, A., Varghese, G. e Walker, D. (2014). P4: programming protocol-independent packet processors. ACM SIGCOMM Computer Communication Review, 44(3):87-95.
- Oliveira, G. W. (2026). Novas Tecnologias em Redes de Comunicação: Redes Definidas por Software (parte II). Notas de aula, Instituto Federal do Paraná, Câmpus Pinhais.
- Open Networking Foundation (2015). OpenFlow Switch Specification, version 1.5.1 (Protocol version 0x06). ONF TS-025.
- Open Networking Foundation (2026). ONOS: Open Network Operating System. Disponível em: <https://opennetworking.org/onos/>. Acesso em: 19 ago. 2026.